11 Graph representation learning and graph neural networks

This chapter covers

- Understanding graph representation learning and its role in scaling machine learning on graphs
- Automating feature engineering with deep learning
- Understanding graph embeddings and their applications
- Working with graph neural networks

In chapters 9 and 10, we explored the fundamental concepts of machine learning (ML) on graphs, demonstrating how these techniques can solve complex tasks like node classification, link prediction, and community detection. We showed how manual feature engineering can effectively capture graph properties and relationships to power downstream ML tasks. These approaches provide insights into what makes graph-based ML work, offering transparency into how our models make decisions.

However, even simple classification tasks require significant effort to design and implement effective features. Manual approaches excel at interpretability and help build intuition, but they face significant challenges when scaled to real-world knowledge graphs (KGs) containing millions of nodes and relationships.

Graph representation learning (GRL) offers a powerful solution to these scalability challenges. Rather than relying on human expertise to design features, GRL uses *deep learning* (DL) techniques to automatically learn optimal representations—called *embeddings*—directly from the graph structure and node attributes. These learned representations capture patterns and similarities that would be difficult or impossible to specify manually. The *embedding process* transforms nodes, edges, and subgraphs into dense vector representations that preserve the graph's structural and semantic properties while being readily consumable by standard ML algorithms.

Graph neural networks (GNNs) have emerged as a particularly effective framework for learning these representations. By iteratively aggregating and transforming information from neighboring nodes, GNNs can automatically discover relevant features for downstream tasks while handling the unique challenges of graph-structured data. This chapter shows how GRL and, in particular, GNNs enable ML on graphs to scale to real-world applications.

We'll explore the technical details of these approaches, but our focus remains practical: understanding when and how to effectively apply these tools to build better intelligent advisor systems. By combining the intuitions developed through manual feature engineering with automated representation learning, we can build graph-powered ML systems that are both scalable and interpretable.

11.1 Embeddings in graph representation learning

When we look at the history of GRL, we can see a fascinating evolution that mirrors the broader development of ML. This field, which focuses on teaching computers to understand and work with graph-structured data, has grown through three generations, each building on the insights and achievements of its predecessors [1]:

1. *The first generation* emerged from traditional mathematics and computer science, focusing on what we now call traditional graph embedding. Imagine trying to take a complex three-dimensional object and create a faithful two-dimensional drawing of it—this was essentially the challenge these early researchers faced. They approached graphs through the lens of classical dimensionality reduction, trying to find ways to represent complex graph structures in simpler, more manageable forms while preserving their essential properties.
2. *The second generation* marked a revolutionary shift, sparked by breakthroughs in natural language processing. Just as word2vec showed us how to capture the meaning of words in numerical vectors, Node2Vec demonstrated that we could do the same with nodes in a graph. Rather than just reducing complexity, we were now capturing meaningful relationships and patterns in our numerical representations.
3. *The third and current generation* has embraced the power of DL, particularly through GNNs. Think of how DL transformed image recognition by automatically learning to identify features from raw pixels—GNNs do something similar

for graphs, automatically learning to understand complex patterns of relationships and interactions.

To harness the power of graph embeddings in real-world applications, we need to understand their fundamental characteristics and how these properties influence their performance in different scenarios. In the following sections, we'll explore the key dimensions that define graph embeddings, from the geometric spaces they occupy to how they capture local and global graph properties.

11.1.1 Understanding graph embeddings: From discrete to continuous

Imagine that you're trying to describe the layout of a city to someone who's never been there. You could list every street and intersection (like a discrete graph representation made by nodes and relationships), or you could draw a map with coordinates for each location (like a continuous embedding made by numbers). The map makes it much easier to understand relationships between locations and do useful calculations about the distance between points [2]. Figure 11.1 depicts the node embedding problem.

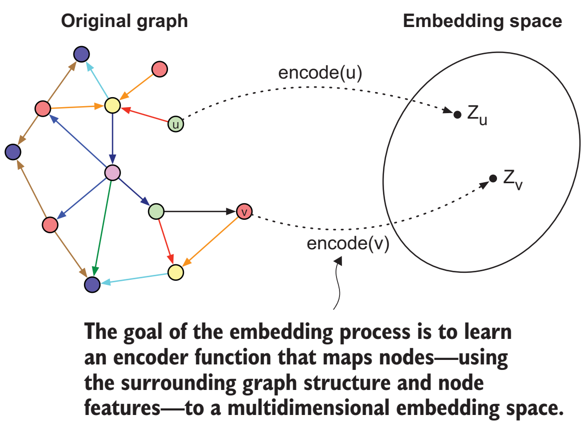

Figure 11.1 The node embedding problem. The goal of the task is to learn an encoding function, encode(x), that maps nodes to a multidimensional embedding space. These embeddings are optimized so that distances in the embedding space reflect relevant aspects of the original graph, such as the relative positions of the nodes.

This is what graph embeddings do: they transform discrete graph structures into continuous vector representations. But why is this transformation so valuable? Let's explore this through the different aspects of embedding approaches.

The geometry of embeddings: Beyond flat spaces

To understand why geometric spaces matter in graph embeddings, we first need to recognize a fundamental challenge in ML: different types of data have different inherent structures. When we work with images or text, the data has a regular, predictable structure. Every pixel in an image has eight neighbors, and every word in a text has one word before and one after it. This regularity allows us to use well-established techniques like convolutional neural networks for images or recurrent neural networks for text [3].

However, in a graph, each node may have a different number of connections and neighborhood structure than its neighbors. This irregularity means we can't directly apply the same techniques we use for regular structured data. We need approaches that can handle this variability, which brings us to the question of geometric spaces.

Figure 11.2 shows the difference between a Euclidean space and a non-Euclidean space. Most ML models, including many graph embedding approaches, operate in Euclidean space: distances are measured with straight lines, and the Pythagorean theorem holds. Euclidean space is particularly effective for data where relationships are uniform and don't change based on position or scale. When we embed something in Euclidean space, we represent it as a vector of coordinates, and the distance between two points is calculated using the standard Euclidean distance formula: the square root of the sum of squared differences between coordinates.

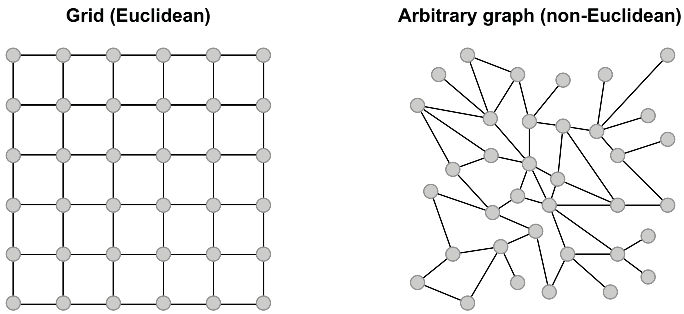

Figure 11.2 Euclidean vs. non-Euclidean space [3]. Pixels in an image are all equally distant from each other and have eight neighbors (except those on the border). In a graph, the number of neighbors can vary enormously from one node to another.

Researchers have discovered that some types of graph structures, particularly hierarchical ones, can be represented more effectively in non-Euclidean spaces, especially hyperbolic spaces. The key difference lies in how space itself behaves: in Euclidean space, the amount of space available grows polynomially as you move outward from a center point (see figure 11.3); but in hyperbolic space, the available space grows exponentially as you move outward, like a tree that branches out more at each level.

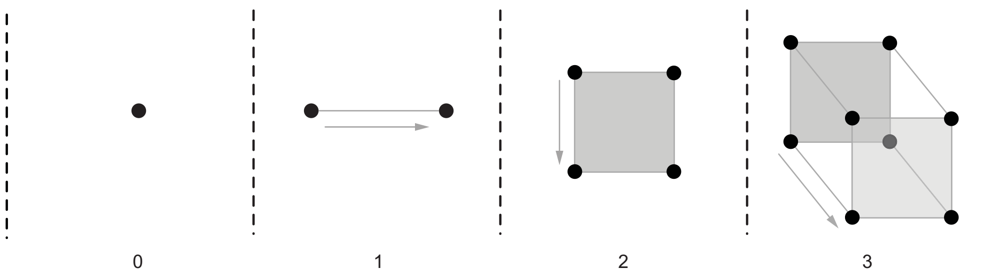

Figure 11.3 Euclidean space grows polynomially as we increase the number of dimensions. Moving from 2D to 3D to 4D and beyond, the available space increases according to a polynomial relationship with the dimension. For example, a unit square in 2D has an area of 1, a unit cube in 3D has a volume of 1, and a unit hypercube in 4D has a hypervolume of 1. Still, the total space available along each axis increases polynomially rather than exponentially.

Consider these practical implications for embeddings. In Euclidean space,

- Vectors are straightforward coordinate points, like (0.5, 0.3).
- Distances grow linearly with coordinate differences.
- The space between points remains constant regardless of their position.
- Most ML models assume this space, making it the default choice for many applications.

In hyperbolic space,

- Vectors are still represented as coordinates, but they behave differently.
- Distances grow exponentially as points approach the boundary of the space.
- The same coordinate difference represents larger distances near the boundary.
- Special distance formulas must be used to calculate similarities between points.

The choice of geometric space becomes particularly important when we know our graph has certain structural properties. For instance, many real-world graphs exhibit a hierarchical structure: think of corporate organizations, biological taxonomies, or internet network topology. In these cases, hyperbolic embeddings may preserve the graph's structure more faithfully than Euclidean embeddings.

It's important to understand that the Euclidean space works well for many applications. We should consider non-Euclidean embeddings only when we have strong evidence that our graph structure would benefit from their properties. When we do use non-Euclidean embeddings, we need to be mindful of how this affects our entire pipeline—from how distances are calculated to how we visualize and interpret the results. This geometric perspective is part of a broader field called *geometric deep learning* (GDL), which studies how to apply DL techniques to data with nonstandard geometric structures. Understanding these geometric spaces helps us choose the right tools for our graph problems and interpret the results appropriately.

LOCAL VS. GLOBAL: UNDERSTANDING POSITIONAL AND STRUCTURAL EMBEDDINGS

The distinction between positional and structural embeddings reflects a fundamental question in graph analysis: what aspects of the graph structure are most important to preserve? Let's explore this through an example of a social network [3]:

- *Positional embeddings* are like preserving the *absolute* positions of people in a social network. If we think of a friend group, positional embeddings would maintain information about who is central to the entire network, who bridges different communities, and how many steps it takes to get from one person to another. These embeddings use techniques like matrix factorization and random walks to capture global properties.
- *Structural embeddings* focus on *relative* positions or local patterns. Two people might be far apart in the network, but if they have similar friendship patterns (like both being the organizers of their respective friend groups), structural embeddings would represent them similarly. This is where GNNs excel, as they can learn to recognize and preserve these local patterns of connectivity.

Positional embeddings, with their focus on preserving global graph structure, have proven valuable for unsupervised tasks where the overall network topology matters most, such as link prediction (where understanding global connection patterns helps predict missing edges) and clustering (where knowing a node's position in the broader network helps identify communities). On the other hand, structural embeddings, particularly those generated by GNNs, excel in supervised tasks. Their ability to capture local neighborhood patterns makes them especially effective for node classification (where similar local structures often indicate similar labels) and whole graph classification (where local patterns can define graph-level properties).

Recent advances in the field have begun to blur these traditional boundaries. New architectures like positional GNNs have emerged, incorporating global positional information into the typically local-focused GNN framework. Furthermore, theoretical work has revealed deeper connections between these two perspectives, suggesting that positional and structural embeddings may capture complementary aspects of graph structure.

Learning strategies: Transductive vs. inductive approaches

The choice between transductive and inductive learning approaches reflects an important question in ML: how should models handle new, unseen data? This distinction is particularly important in graph learning, where new nodes and edges frequently appear [3]:

- *Transductive learning* is like learning to solve a specific puzzle: you can get very good at it, but your knowledge may not transfer to a different puzzle. In graph terms, transductive approaches learn embeddings for a fixed set of nodes, optimizing them directly for the known graph structure. This works well for static graphs where you don't expect new nodes to appear. Transductive methods allow us to infer new information between nodes analyzed during training. For example, when working with partially labeled nodes, we can classify unlabeled nodes based on the patterns learned from the known structure. Similarly, we can predict new edges between graph nodes that were observed during the training process.
- *Inductive learning* is like learning the general strategies for solving puzzles. Instead of memorizing specific patterns, inductive approaches learn rules that can be applied to new situations. This makes them useful for dynamic graphs. Node features are used in inductive GRL methods to learn embeddings with parametric mappings. The learning goal is achieved by optimizing such parametric mappings instead of directly optimizing the embeddings. This means the learning mappings can be applied to any node, even those that were not seen during training.

Consider a recommendation system for an e-commerce platform. A transductive approach works well for recommending existing products to existing users based on their past interactions. However, when a new user joins the platform (or a new product is offered), we need an inductive approach that can generate meaningful embeddings for this new user based on their initial interactions and profile information.

THE ROLE OF SUPERVISION IN LEARNING EMBEDDINGS

The distinction between supervised and unsupervised learning in graph embeddings reflects different levels of available information and different goals for the embedding process:

- *Unsupervised learning* in graph embeddings is like trying to understand the social dynamics of a group by watching who talks to whom, without knowing anything about the content of their conversations. The goal is to discover natural patterns and structures in the graph data. This approach relies on the assumption that the graph structure alone contains valuable information—which is often true!
- *Supervised learning* is like having additional context about the interactions we observe. This additional information helps guide the learning process toward specific goals.

11.1.2 Real-world applications and examples

Let's explore how these aspects of graph embeddings come together in real-world applications:

- *Social network analysis*—Consider a large social network like LinkedIn. Here, structural embeddings identify users with similar professional roles across different industries, while positional embeddings help understand who the key influencers and bridge-builders are in the network. The platform needs inductive learning capabilities to handle new users and may use supervised learning with job titles and skills to improve recommendation accuracy.
- *Biological networks*—In protein interaction networks, the choice of geometry is critical. The hierarchical organization of biological systems often benefits from hyperbolic embeddings. The need to constantly incorporate new discoveries makes inductive learning essential, and the availability of experimental data enables supervised learning approaches to improve prediction accuracy.
- *Medical knowledge graphs*—Consider a KG representing medical knowledge. The hierarchical nature of medical terminology (from broad categories to specific conditions) makes hyperbolic embeddings particularly appropriate. Structural embeddings help identify similar concepts across different branches of medicine, and supervised learning with expert-labeled data helps ensure accurate relationships between concepts.

Through these examples, we can see how different embedding approaches combine to solve real-world problems. The choice of embedding strategy isn't just a technical decision—it reflects our understanding of the problem structure and our goals for the solution.

11.2 The encoder–decoder model

The encoder–decoder model provides a powerful and unified way to understand how graph embedding methods work. Think of it like a translation system: first the model converts (encodes) graph information from one form into more compact, standardized representations, and then it translates (decodes) those representations to reconstruct meaningful graph properties [2] (see figure 11.4). In the context of graphs, this means transforming the discrete graph structure into continuous vector representations that preserve the important characteristics of the original graph.

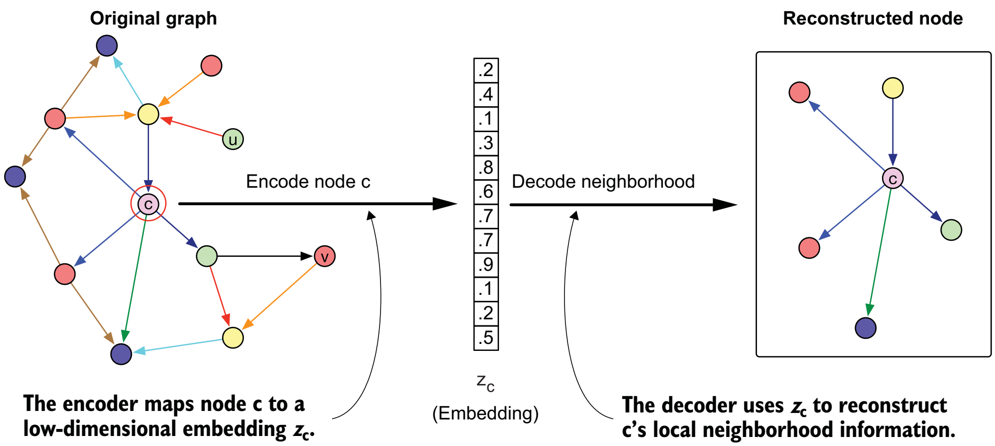

Figure 11.4 Overview of the encoder–decoder approach. The encoder maps node c to a low-dimensional embedding z_c. The decoder then uses z_c to reconstruct c's local neighborhood information.

11.2.1 The encoder: Converting graph structure to vectors

The encoder takes the raw graph data—its structure and any additional features—and converts it into dense vector representations (embeddings) for each node. The encoder takes two key inputs:

- The graph structure, represented as an adjacency matrix that shows how nodes are connected
- Node features, which provide additional information about each node's characteristics

What makes the encoder interesting is its flexibility: different methods can encode different aspects of the graph. Some focus on preserving the global structure and relative positions of nodes, whereas others prioritize capturing local neighborhood patterns. The encoder can be as simple as a lookup table that assigns each node a vector, or as sophisticated as a neural network that processes both structure and features.

11.2.2 The decoder: Reconstructing graph properties

The decoder takes the embeddings created by the encoder and attempts to reconstruct important properties of the original graph. This reconstruction ensures that the embeddings have captured meaningful information. The decoder's task varies depending on what aspects of the graph we want to preserve:

- For methods focused on graph structure, the decoder may try to predict whether nodes are connected in the original graph.
- To capture node similarity, it may reconstruct measures of neighborhood overlap between nodes.
- In cases with additional supervision, the decoder can predict node labels or other properties.

This reconstruction task serves as a training signal. By trying to minimize the difference between the decoder's predictions and the actual graph properties, we can optimize the encoder to learn better representations.

11.2.3 The power of the framework

What makes the encoder–decoder framework particularly valuable is its ability to unify many different approaches to GRL. Whether we're looking at matrix factorization methods, random walk-based approaches, or GNNs, they can all be understood as different ways of implementing this basic encode–decode pattern. The framework also helps us understand the trade-offs involved in different methods. For instance, simpler encoders may be more computationally efficient but capture less complex patterns, whereas more sophisticated neural network-based encoders can learn richer representations but require more data and computation to train effectively. The encoding–decoding perspective provides a lens through which we can analyze, compare, and improve methods for learning representations of graph-structured data.

11.2.4 Node2Vec: An example of an encoder–decoder framework

Consider the karate club network we worked with in chapter 9, where nodes represent club members and edges represent friendships between them. The club split into two groups following a dispute between the instructor (node 0) and the club administrator (node 33). This makes it a perfect example to understand how Node2Vec [4] works within the encoder–decoder framework.

THE ENCODING PROCESS IN NODE2VEC

First, Node2Vec's encoder needs to transform each club member (node) into a numerical vector that captures their position and role in the friendship network. Here's how it does this:

1. *Random walk generation*—The encoder starts by taking "walks" through the friendship network. Let's say we start at the instructor (node 0). A walk might look like $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$, representing a path through friends of friends. Node2Vec can adjust how it explores the network by balancing between going

broad (exploring different social circles in the club) and going deep (staying within tight-knit groups of friends).

2. *Vector creation*—Based on these walks, the encoder creates a vector for each member that captures their social context. Members who frequently appear near each other in these walks (close friends or in the same social circles) will get similar vector representations.

The decoding process

The decoder takes these vectors and reconstructs information about the friendship network. For any two members i and j, the decoder tries to predict how likely they are to be friends based on their vector representations. It does this using the *softmax function*, which essentially asks, "Given member i's vector, how likely are we to see member j nearby in our random walks?"

This encoding–decoding process reveals interesting patterns in the karate club network:

- *Community structure*—Members who ended up in the same faction after the split tend to get similar vector representations. The vectors capture not just direct friendships but also broader community affiliations.
- *Bridge members*—Members with friends in both factions get vector representations that reflect their intermediate position. These vectors help identify potential mediators in the conflict.
- *Leadership roles*—The instructor and administrator get distinct vector representations that reflect their central but opposing positions in the network. Their vectors capture their roles as community leaders through their connection patterns.

The power of the framework

The encoder–decoder framework helps us understand why Node2Vec works:

- The encoder's random walks capture both local friendships and broader social structures.
- The decoder's prediction task ensures that the vectors preserve meaningful social relationships.
- Together, they create representations that can predict both friendships and faction membership.

This example shows how the encoder–decoder framework helps us understand and improve real-world network analysis. The following listing shows how to apply Node2Vec to the karate club network.

Listing 11.1 Applying Node2Vec embedding to the karate club network

```
import networkx as nx
import numpy as np
from node2vec import Node2Vec
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
```

```
G = nx.karate_club_graph()                          <- Loads the karate
                                                       club network
faction_labels = {
    0: 0, 1: 0, 2: 0, 3: 0, 4: 0, 5: 0, 6: 0, 7: 0, 8: 0,
    9: 1, 10: 1, 11: 0, 12: 0, 13: 0, 14: 1, 15: 1, 16: 0,
    17: 0, 18: 1, 19: 0, 20: 1, 21: 0, 22: 1, 23: 1, 24: 1,
    25: 1, 26: 1, 27: 1, 28: 1, 29: 1, 30: 1, 31: 1, 32: 1,
    33: 1
}                                                   <- Defines faction labels
                                                       (0 for instructor,
                                                       1 for administrator)
nx.set_node_attributes(G, faction_labels, 'faction')  <- Assigns faction attributes
                                                         to network nodes
node2vec = Node2Vec(
    G,
    dimensions=16,
    walk_length=10,
    num_walks=20,
    p=1,
    q=1,
    workers=4
)                               <- Configures Node2Vec with
                                   embedding parameters

model = node2vec.fit(window=5)  <- Trains the Node2Vec model

def decode_similarity(model, node1, node2):
    return model.wv.similarity(str(node1), str(node2))  <- Computes similarity
                                                           between node
                                                           embeddings

def visualize_graph_and_embeddings(model, G):
    embeddings = np.zeros((len(G.nodes()), model.vector_size))
    for i, node in enumerate(G.nodes()):
        embeddings[i] = model.wv[str(node)]

    tsne = TSNE(n_components=2, random_state=42)
    node_pos_2d = tsne.fit_transform(embeddings)

    fig, axes = plt.subplots(1, 2, figsize=(18, 8))

    # Left plot: original graph
    pos = nx.spring_layout(G, seed=42)
    colors = ['red' if G.nodes[node]['faction'] == 0 else 'blue'
              for node in G.nodes()]
    nx.draw(
        G,
        pos,
        ax=axes[0],
        with_labels=True,
        node_color=colors,
        node_size=500,
        font_size=10
    )
    axes[0].set_title("Original Karate Club Graph")

    # Right plot: node embeddings
    axes[1].scatter(node_pos_2d[:, 0], node_pos_2d[:, 1], c=colors, s=100)
    for i, node in enumerate(G.nodes()):
        axes[1].annotate(str(node), (node_pos_2d[i, 0],
            node_pos_2d[i, 1]), fontsize=9)
    axes[1].set_title("Node2Vec Embeddings (t-SNE)")
```

```
    plt.tight_layout()
    plt.show()                                      Visualizes the original graph and
                                                    node embeddings in 2D space

print("Similarity between instructor (0) and their close ally (1):",
      decode_similarity(model, 0, 1))
print("Similarity between instructor (0) and administrator (33):",
      decode_similarity(model, 0, 33))                    Prints similarity scores
                                                          between key nodes
visualize_graph_and_embeddings(model, G)        Generates and
                                                displays an
def analyze_community_structure(model, G):      embedding
    instructor_allies = []                      visualization
    administrator_allies = []

    for node in G.nodes():
        if node not in [0, 33]:
            sim_to_instructor = decode_similarity(model, node, 0)
            sim_to_administrator = decode_similarity(model, node, 33)

            if sim_to_instructor > sim_to_administrator:
                instructor_allies.append(node)          Analyzes the community
            else:                                       structure based on embeddings
                administrator_allies.append(node)

    return instructor_allies, administrator_allies     Prints predicted
                                                         faction assignments
instructor_group, administrator_group = analyze_community_structure(model, G)
print("\nPredicted instructor's faction:", sorted(instructor_group))
print("Predicted administrator's faction:", sorted(administrator_group))
```

This implementation demonstrates the aspects of the encoder–decoder framework that we just discussed. Run it! The t-SNE visualization shows how the encoded vectors cluster nodes by faction, and the similarity calculations between some important nodes demonstrate how the decoded embeddings can predict relationships in the original graph. You can see how members close to the instructor (node 0) have different embedding patterns than those close to the administrator (node 33), reflecting the real social dynamics that led to the club's split.

11.3 Shallow embeddings: A first approach to graph representation

The simplest way to understand graph embeddings is through *shallow embeddings*. These represent the most straightforward implementation of the encoder–decoder framework, but their simplicity comes with important limitations that have driven the development of more sophisticated approaches [2].

11.3.1 Understanding shallow embeddings

In shallow embedding methods, the encoder performs a simple operation: it acts as a lookup table, directly mapping each node to its vector representation. Think of it like assigning each person in a social network a unique "coordinate" in a high-dimensional space. This is similar to how a dictionary works: each word (node) has its own dedicated entry (vector).

In the encoder–decoder framework, this approach can be understood as follows:

- The encoder maintains a matrix where each row corresponds to a node's embedding vector. When asked to encode a node, it returns the corresponding row from this matrix. This is why we call these embeddings "shallow"—there's no deep processing or transformation of the input, just a direct lookup operation.
- The decoder takes these embedding vectors and tries to reconstruct important properties of the graph, such as whether two nodes are connected or how similar their neighborhoods are. This reconstruction process helps optimize the embedding vectors during training to capture meaningful patterns in the graph structure.

Let's consider a concrete example. In the karate club network we discussed earlier, a shallow embedding approach would create a separate vector for each club member. These vectors would be adjusted during training so that members who interact frequently end up with similar vectors and those who rarely interact end up with dissimilar vectors.

11.3.2 Limitations of shallow embeddings

Although shallow embeddings have achieved success in many applications, they face several important limitations:

- *Parameter inefficiency*—Shallow embeddings require a separate vector for each node in the graph. This means the number of parameters grows linearly with the number of nodes, making them impractical for very large graphs. For instance, in a social network with millions of users, we would need millions of embedding vectors.
- *No parameter sharing*—Shallow embeddings don't share parameters between nodes. This means they can't use common patterns or structures that may appear in different parts of the graph. In the karate club example, if two members play similar roles in different social circles, the model has to learn these patterns independently for each member rather than recognize and reuse the pattern.
- *Feature blindness*—These methods don't take advantage of node features or attributes. If we knew additional information about each club member (like age, interests, or role), shallow embeddings wouldn't have a natural way to incorporate this information into the representations.
- *Transductive nature*—Shallow embeddings are inherently transductive: they can only generate embeddings for nodes that were present during training. If a new member joins the club, we can't automatically generate an embedding for them because they weren't part of the original lookup table. We would need to retrain the entire model to incorporate new nodes.

These limitations have motivated the development of more sophisticated approaches, particularly GNNs, which we'll explore in section 11.6. These advanced methods learn to generate embeddings based on both graph structure and node features, enabling inductive learning and better parameter sharing.

Despite their limitations, shallow embeddings remain important both historically and practically. They help us understand the fundamental challenges of GRL and provide a baseline against which we can compare more sophisticated approaches. Their simplicity also makes them useful in situations where the graph is relatively small and static, and when computational resources are limited.

11.4 Embeddings in knowledge graphs

In our previous sections, we explored how to learn embeddings for simple graphs where edges represent a single type of relationship between nodes. However, real-world graphs often have much richer structures, in particular those we are addressing in this book for implementing an intelligent advisor system. Think of a biomedical KG where nodes represent drugs, diseases, genes, and proteins, with dozens of different types of relationships connecting them. These multirelational graphs require more sophisticated approaches to embedding.

In this section, we'll extend our discussion of shallow embeddings to handle these complex KGs. We need to introduce new techniques to handle the multiple types of relationships; this is particularly important for tasks like KG completion, where we predict missing relationships between entities.

Consider a biomedical KG in which we have *entities* (*Aspirin*, *Inflammation*, *Headache*, *COX-2*) and *relationships* (`TREATS`, `INHIBITS`, `CAUSES`). In a simple graph, we may just care about whether entities are connected. However, in a KG, the type of connection matters significantly. We need to be able to capture the following:

- (*Aspirin*, `TREATS`, *Headache*)
- (*Aspirin*, `INHIBITS`, *COX-2*)
- (*Inflammation*, `CAUSES`, *Headache*)

Our embedding approach must encode not just whether entities are related but also *how* they are related. So, we need a way to measure how well our embeddings capture the graph structure (the loss function) and a way to handle different types of relationships between nodes (the multirelational decoder). Let's explore how these components work together to create effective graph representations.

11.4.1 Loss function

Think of a *loss function* as a teacher grading a student's work. Just as a teacher needs clear criteria to evaluate performance, we need ways to measure how well our embeddings capture the graph's structure. However, designing an effective loss function for graph embeddings presents some challenges.

The simplest approach may seem to be directly comparing our predicted connections with the actual graph structure using something like mean squared error. But this approach faces two practical limitations:

- *Computational efficiency*—In a graph with millions of nodes, comparing every possible pair of nodes becomes computationally impossible. For example, in a social network with 1 million users, we'd need to evaluate nearly a trillion potential connections.
- *Sparsity*—Most real-world graphs are sparse, meaning most nodes aren't connected to each other. A social media user may have hundreds of friends, but this is tiny compared to the millions of total users. Our loss function needs to handle this imbalance effectively.

To address these challenges, modern approaches typically use *negative sampling with cross-entropy loss*. Here's how this loss function works:

$$\mathcal{L} = \sum_{(u,\tau,v)\in\mathcal{E}} -\log(\sigma(\mathrm{DEC}(z_u,\tau,z_v))) - \gamma\mathbb{E}_{v_n\sim p_{n,v}(v)}[\log(\sigma(-\mathrm{DEC}(z_u,\tau,z_{v_n})))]$$

Let's understand each element.

- Basic components:
  - $\mathcal{L}$—The total loss we're trying to minimize
  - Σ—Sum over all existing edges in our KG
  - σ—The sigmoid function, which converts scores to probabilities between 0 and 1
  - DEC—Our decoder function that scores how likely a relationship is
  - γ—A balancing parameter (gamma) that controls the importance of negative samples
- Node and relationship elements:
  - u—The head node (e.g., "Aspirin")
  - τ (tau)—The relationship type (e.g., "TREATS")
  - v—The tail node (e.g., "Headache")
  - z_u—The embedding vector for node u
  - z_v—The embedding vector for node v
- The positive part:

$$-\log(\sigma(\mathrm{DEC}(z_u,\tau,z_v)))$$

  - $\mathrm{DEC}(z_u, \tau, z_v)$—Computes a score for the true relationship
  - $\sigma(\ldots)$—Converts this score to a probability
  - $-\log(\ldots)$—Makes the loss larger when we give low probabilities to true relationships

- The negative part:

$$-\gamma \mathbb{E}_{v_n \sim p_{n,v}(v)}[\log(\sigma(-\mathrm{DEC}(z_u, \tau, z_{v_n})))]$$

  – v_n—A negative sample (e.g., *COX-2* when sampling alternatives for *Headache*)
  – $P_{n,v}(V)$—The distribution we use to sample negative examples
  – $\mathbb{E}$—Expected value over negative samples
  – –DEC(...)—Negative score for false relationships
  – γ—Multiplier to control the importance of negative samples

Let's look at a practical example. Suppose we're trying to learn embeddings for our biomedical KG. We have a fact: "aspirin treats headache." Our loss function needs to do the following:

- *Reward true facts.* The first term encourages the decoder to give a high score to real relationships. In our example, it should give a high score to (*Aspirin*, `TREATS`, *Headache*).
- *Penalize false facts.* The second term involves "negative sampling"—we randomly sample entities that aren't known to have this relationship. For example, we may sample (*Aspirin*, `TREATS`, *COX-2*) and (*Aspirin*, `TREATS`, *Inflammation*).

The loss function encourages the decoder to give low scores to these negative samples.

The parameter γ balances these two objectives. Think of it as deciding how much we care about correctly identifying false relationships compared to correctly identifying true ones. In practice, because KGs are usually sparse (most possible relationships don't exist), we typically want $\gamma > 1$ to emphasize correctly identifying false relationships.

This loss function is computationally efficient because instead of checking all possible relationships, we only need to look at the true relationships and a small sample of negative ones. In practice, we may use 5–10 negative samples for each true relationship.

The art of negative sampling in KGs

The effectiveness of negative sampling depends heavily on how we choose these negative examples. At its core, negative sampling is about teaching our model to distinguish between relationships that should exist and those that shouldn't. However, this isn't as straightforward as it may seem.

The simplest approach is to randomly sample nodes from the graph to create negative examples. For instance, if we have the true relationship (*Aspirin* `TREATS` *Headache*), we might create negative samples like (*Aspirin* `TREATS` *Laptop*) and (*Aspirin* `TREATS` *Mountain*). This approach is computationally efficient, but it has two main limitations:

- *False negatives*—We may accidentally sample relationships that exist in our KG. Some systems address this by checking and filtering out such accidental true relationships.

- *Overly simple examples*—Many random samples are obviously wrong and don't help our model learn meaningful patterns.

To address these limitations, researchers have developed more sophisticated sampling strategies:

- *Type-constrained sampling*—Instead of sampling any random node, we only sample nodes that make semantic sense for the relationship. For example, when creating negative samples for a `TREATS` relationship, we only sample diseases as potential targets, not arbitrary entities. This forces the model to learn more subtle distinctions.
- *Adversarial sampling*—We generate challenging negative examples. Instead of random sampling, we identify examples that are likely to be confused with true relationships, helping the model develop a more nuanced understanding.

We can also sample negative examples by replacing either the subject or object of a relationship (or both). For example, given "Aspirin TREATS Headache," we could create negative samples by:

- Replacing the treatment—"Sunlight TREATS Headache"
- Replacing the condition—"Aspirin TREATS Happiness"
- Replacing both—"Sunlight TREATS Happiness"

In practice, considering replacing either the source or the destination helps prevent biases, especially in KGs where relationship direction matters.

11.4.2 Multirelationship decoder

Once we have our loss function, we need decoders that can handle different types of relationships. These decoders need to capture various patterns that appear in KGs:

- *Symmetric vs. asymmetric relations*—Some relationships, like `similar_to`, are symmetric (if A is similar to B, then B is similar to A). Others, like `causes`, are asymmetric (if A causes B, it doesn't mean B causes A).
- *Compositional patterns*—Sometimes relationships can be composed. For example, if drug A treats disease B, and disease B is a type of disease C, we may infer that drug A treats disease C.
- *Inverse relations*—Some relationships have natural inverses. If A `contains` B, then B is `part_of` A.

Let's look at three main approaches to building such decoders:

- *Translation-based* (TransE [5])—This is one of the most intuitive approaches. Each relationship is represented as a vector that "translates" one entity into another. In our medical example, if we add the `TREATS` vector to the *Aspirin* vector, we should get close to the *Headache* vector.
- *Matrix-based* (RESCAL [6])—This expressive approach represents each relationship as a matrix that transforms entity vectors. Although powerful, it requires

more parameters: for a graph with 1,000 entities and 100 relationship types, we'd need millions of parameters just for the relationships.

- *Semantic matching* (DistMult [7], ComplEx [8])—These decoders measure similarity between entities while taking the relationship type into account. ComplEx is particularly interesting because it uses complex numbers to handle asymmetric relationships elegantly.

The choice of decoder significantly impacts what kinds of patterns your model can learn. TransE, for example, is great at capturing compositional patterns but struggles with many-to-one relationships. ComplEx handles asymmetric relationships well but may not capture compositional patterns as effectively.

This framework of sophisticated loss functions and multirelational decoders forms the foundation for modern KG embeddings. In the next sections, we'll see how these ideas extend to even more powerful approaches using GNNs.

11.5 Message passing and graph neural networks

Although shallow embeddings provide an effective starting point for learning graph representations, they face limitations when dealing with large-scale graphs or when we need to capture complex patterns of relationships. GNNs offer a more sophisticated approach through a mechanism called *neural message passing*, which allows nodes to iteratively learn from their neighborhoods. Let's explore how this powerful framework works.

11.5.1 The message-passing framework: A neural conversation

Think of message passing in a GNN as a sophisticated conversation happening across the graph. Just as humans share and process information through conversations with their social circles, nodes in a graph share and update information through structured interactions with their neighbors. This "neural conversation" happens in rounds, with each round allowing information to travel one step further through the graph.

The key insight of message passing is that nodes can learn not just from their immediate connections but also from the broader graph structure through iterative updates. During each iteration of message passing, every node does the following:

1. Collects messages from its neighbors
2. Processes these messages to extract relevant information
3. Updates its own representation based on the processed messages

This process is formalized through two key functions:

- `AGGREGATE`—Collects and combines messages from neighboring nodes
- `UPDATE`—Uses the aggregated messages to update the node's representation

Figure 11.5 summarizes this concept [2].

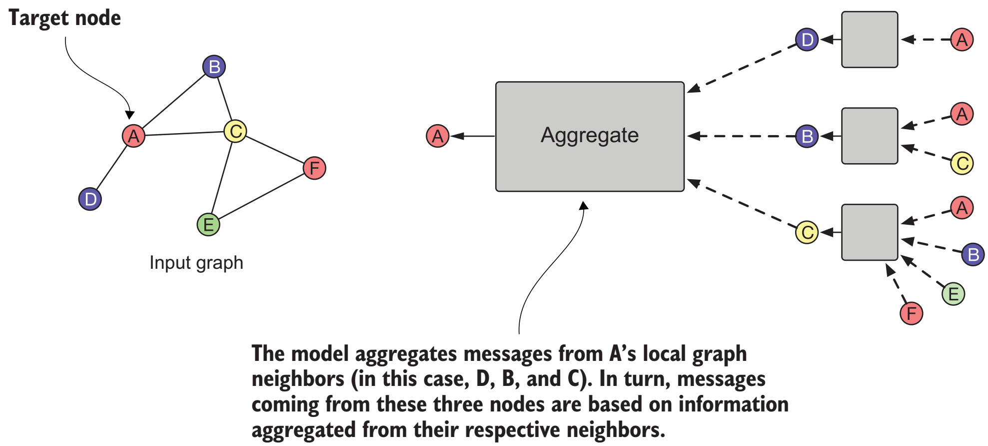

Figure 11.5 Example of how a single node aggregates messages from its local neighborhood in the message-passing model. Notice that the computation graph of the GNN forms a tree structure by unfolding the neighborhood around the target node.

For example, in a social network analyzing user interests, the message-passing process may work like this:

1. A user's initial representation includes their explicitly stated interests.
2. In the first round, they receive messages about their friends' interests.
3. In the second round, they get information about their friends-of-friends' interests.
4. After several rounds, each user's representation captures patterns of interests in their broader social circle.

11.5.2 Motivation and intuition: Why message passing works

The power of message passing comes from its ability to capture both local and global patterns in the graph structure. After k iterations of message passing, each node's representation contains information from its k-hop neighborhood. This progressive gathering of information serves two important purposes:

- *Structural information*—The message-passing process naturally encodes the graph's topology. For example, in a molecular graph, after several rounds of message passing, an atom's representation may encode information about being part of a benzene ring or other molecular structures.
- *Feature-based information*—Message passing also allows nodes to learn from the features of their neighbors. In a citation network, for instance, a paper's representation will gradually incorporate information from related papers, helping us better understand its place in the broader academic landscape.

11.5.3 The basic GNN model

Let's look at how we can implement this message-passing framework in practice. The basic GNN model defines the update rule for each node u at iteration k as

$$h_u^{(k)} = \sigma\left(W_{\text{self}}^{(k)} h_u^{(k-1)} + W_{\text{neigh}}^{(k)} \sum_{v \in \mathcal{N}(u)} h_v^{(k-1)} + b^{(k)}\right)$$

where

- $h_u^{(k)}$ represents node u's representation at iteration k.
- $W_{\text{self}}{}^{(k)}$ and $W_{\text{neigh}}{}^{(k)}$ are trainable parameter matrices.

NOTE These parameters can be shared across GNN message-passing iterations or trained separately for each layer (iteration).

- σ is a nonlinear activation function (like ReLU or tanh).
- The summation $\Sigma_{v \in \mathcal{N}(n)}\, h_v^{(k\text{-}1)}$ runs over all neighbors v of node u.
- $b^{(k)}$ is the bias term, which is often omitted (if used, it can be shared across multiple iterations or trained differently for each layer).

This basic model is analogous to a standard neural network layer but operates on graph-structured data. The primary difference is that instead of processing fixed-size inputs, it can handle varying numbers of neighbors through the summation operation.

We can define the basic GNN formula by splitting the two components of the `AGGREGATE` and `UPDATE` functions to match the mental model:

$$m_{\mathcal{N}(u)}^{(k-1)} = \text{AGGREGATE}^{(k-1)}\left(\left\{h_v^{(k-1)}, \forall\, v \in \mathcal{N}(u)\right\}\right) = \sum_{v \in \mathcal{N}(u)} h_v^{(k-1)}$$

$$\text{UPDATE}\left(h_u^{(k)}, m_{\mathcal{N}(u)}^{(k-1)}, b^{(k)}\right) = \sigma\left(W_{\text{self}}^{(k)} h_u^{(k-1)} + W_{\text{neigh}}^{(k)} m_{\mathcal{N}(u)}^{(k-1)} + b^{(k)}\right)$$

11.5.4 Message passing with self-loops

An important variation of the basic neural message-passing framework adds self-loops to the graph. In this approach, instead of having separate `AGGREGATE` and `UPDATE` steps, we treat each node as its neighbor during the aggregation process. This simplifies the message-passing equation to

$$h_u^{(k)} = \text{AGGREGATE}^{(k-1)}\left(\left\{h_v^{(k-1)}, \forall\, v \in \mathcal{N}(u) \cup \{u\}\right\}\right)$$

The self-loop approach offers several advantages:

- Simpler implementation due to combining the `AGGREGATE` and `UPDATE` steps
- Often helps prevent overfitting through parameter sharing
- Can improve model stability during training

However, this simplification comes with a trade-off: by treating a node's information the same as its neighbors', we lose flexibility in how nodes can combine their current state with neighborhood information.

The choice between standard message passing and the self-loop variant often depends on the specific application and the type of patterns we want our model to learn. For tasks where the distinction between a node's features and its neighbors' features is critical, the standard approach may be more appropriate. For tasks where this distinction is less important, the self-loop variant can offer a simpler and more robust solution. Chapter 12 includes examples that use this approach instead of `AGGREGATE` and `UPDATE`.

11.6 Generalized aggregation and update methods

The message-passing framework we explored earlier provides a powerful mental model for understanding how GNNs process and learn from graph-structured data and feature information. However, this framework can be significantly enhanced by introducing specialized methods for aggregating neighborhood information and updating node representations. In this section, we'll explore these generalizations and how they can improve GNN performance.

Just as traditional neural networks have evolved from simple perceptrons to sophisticated architectures with skip connections, attention mechanisms, and normalization layers, GNNs can benefit from similar architectural innovations. The challenge is adapting these ideas to respect the unique properties of graph-structured data, particularly the irregular connectivity patterns and varying neighborhood sizes that characterize real-world graphs.

Architectural enhancements for GNNs

Skip connections, also known as *residual connections*, create shortcuts in neural networks by allowing information to bypass one or more layers. In GNNs, skip connections help preserve node features from earlier layers, preventing the over-smoothing problem where node representations become too similar after multiple message-passing steps. They enable deeper GNN architectures by maintaining gradient flow during training.

Attention mechanisms allow a model to dynamically focus on different parts of the input by assigning varying importance weights. In GNNs, attention mechanisms help nodes selectively aggregate information from their neighbors by weighting neighbor contributions differently, based on their relevance to the current task. This enables more sophisticated message passing that can capture complex node relationships.

Normalization layers help stabilize neural network training by controlling the distribution of layer outputs. In GNNs, normalization is particularly important because nodes can have widely varying numbers of neighbors. Layer normalization and batch normalization help manage these differences by scaling features appropriately, leading to more stable training and better model convergence.

Let's examine two areas where the basic GNN framework can be enhanced: neighborhood aggregation and node-state updates. These enhancements address common challenges in graph learning, such as capturing complex neighborhood patterns, handling varying neighborhood sizes, and maintaining stable training dynamics.

11.6.1 Neighborhood normalization

A fundamental challenge in processing graph-structured data is handling neighborhoods of different sizes. A naive approach to aggregating neighborhood information can lead to numerical instabilities and make it difficult for the model to learn effectively across varying scales.

Neighborhood normalization techniques address this challenge by scaling the aggregated information based on neighborhood properties. The simplest form of normalization is to take the mean of neighbor features rather than their sum. The following formula performs the aggregation by computing the mean:

$$m_{\mathcal{N}(u)}^{(k-1)} = \frac{\sum_{v \in \mathcal{N}(u)} h_v^{(k-1)}}{|\mathcal{N}(u)|}$$

where $h_v^{(k\text{-}1)}$ is the feature vector coming from the neighbor v, and the denominator $|\mathcal{N}(u)|$ represents the number of neighbors of u. The sum operates on vectors, so each item is summed with the others in the same position.

More sophisticated normalization schemes can offer better performance. For example, the symmetric normalization approach popularized by Kipf and Welling [9] in their graph convolutional network uses the following formula:

$$m_{\mathcal{N}(u)}^{(k-1)} = \sum_{v \in \mathcal{N}(u)} \frac{h_v^{(k-1)}}{\sqrt{|\mathcal{N}(u)|\,|\mathcal{N}(v)|}}$$

This symmetric normalization is particularly effective in scenarios where both the source and target nodes' degrees should influence the message strength. For instance, in citation networks, this normalization reduces the impact of highly cited papers that may otherwise dominate the message-passing process. A paper cited thousands of times across diverse fields shouldn't necessarily have a stronger influence than more specialized, relevant citations.

The choice of a normalization scheme can significantly impact model performance. Consider these practical implications:

- *Mean normalization* (simple averaging) helps ensure that nodes with different numbers of neighbors can be compared on a similar scale.
- *Symmetric normalization* accounts for both sender and receiver node properties, which can be important in directed graphs or graphs where node importance varies significantly.

- Some applications may benefit from learned normalization factors that the model can adjust during training.

On the other hand, after normalization, it can be hard to use the learned embedding to distinguish between nodes of different degrees, making the normalization a lossy operation. Usually, normalization is most helpful in tasks where node feature information is far more useful than structural information or where there is a very wide range of node degrees that can lead to instabilities during optimization.

11.6.2 Neighborhood attention

Normalization helps handle varying neighborhood sizes, but it treats all neighbors equally within their normalized contributions. However, not all relationships in a graph carry equal importance. For example, in a social network, some friendships may be more relevant than others for predicting a user's interests. This is where attention mechanisms come into play.

Attention allows a GNN to learn which neighbors are most relevant for a given task by assigning learnable weights to different neighborhood relationships. The first GNN model to apply this style of attention was Veličković et al.'s graph attention network [10], which uses attention weights to define a weighted sum of the neighbors:

$$m_{\mathcal{N}(u)}^{(k-1)} = \sum_{v \in \mathcal{N}(u)} \alpha_{u,v} h_v^{(k-1)}$$

where $\alpha_{w,v}$ denotes the attention on neighbor $v \in \mathcal{N}(u)$ when we are aggregating information at node u. In the original paper, the attention weights are defined as

$$\alpha_{u,v} = \frac{\exp\left(a^T \left[W h_u \bigoplus W h_v\right]\right)}{\sum_{v' \in \mathcal{N}(u)} \exp\left(a^T \left[W h_u \bigoplus W h_{v'}\right]\right)}$$

where **a** is a trainable attention vector, W is a trainable matrix, and $\oplus$ denotes the concatenation operation.

The GraphSAGE framework [11] provides a practical example of how attention can be integrated into the aggregation process. This implementation uses attention to weight the importance of different neighbors during the aggregation step.

Attention mechanisms can be particularly valuable when the importance of neighbors varies significantly based on the task or context, when the graph contains noisy or irrelevant connections, or if the model needs to capture complex, nonlinear relationships between nodes.

11.6.3 Multihead attention and transformer connections

The attention mechanisms we've discussed share deep connections with transformer architectures that have revolutionized natural language processing and, more recently, become the backbone of LLMs. Understanding these connections can provide valuable insights into both the design and capabilities of modern GNNs.

Just as transformers process sequences of words by allowing each word to attend to all other words in a sentence, graph attention networks allow each node to selectively attend to its neighbors. However, a single attention mechanism may not be sufficient to capture all relevant patterns in the data. This insight led to the development of multihead attention in both domains.

In the context of GNNs, multihead attention works by maintaining several independent attention mechanisms that operate in parallel. Each attention head can learn to focus on different aspects of the neighborhood relationships. For instance, in a molecular graph, one head may learn to focus on chemical bond types while another captures spatial arrangements. Here's how this can be implemented.

Listing 11.2 Multihead attention implementation

```
class MultiHeadGraphAttention(nn.Module):
    def __init__(self, input_dim, num_heads):
        super().__init__()
        self.num_heads = num_heads
        self.head_dim = input_dim
        self.attention_heads = nn.ModuleList([
            GraphAttentionHead(self.head_dim)
            for _ in range(num_heads)
        ])

    def forward(self, node_features, neighbor_features):
        node_chunks = torch.chunk(node_features,
                                  self.num_heads, dim=-1)
        neighbor_chunks = torch.chunk(neighbor_features,
                                      self.num_heads, dim=-1)
        head_outputs = []
        for i, head in enumerate(self.attention_heads):
            head_out = head(node_chunks[i], neighbor_chunks[i])
            head_outputs.append(head_out)

        return torch.cat(head_outputs, dim=-1)
```

Initializes multiple attention heads, each processing a slice of the input dimension

Splits node features into chunks for parallel processing by different heads

Splits neighbor features into chunks for parallel processing by different heads

Applies each attention head to its corresponding feature chunk

Concatenates outputs from all heads to form the final node representation

The parallel with transformer architecture becomes even clearer when we consider the fundamental operation in both cases. In transformers, the attention mechanism computes query (`Q`), key (`K`), and value (`V`) matrices to determine attention weights, as shown in the following listing.

Listing 11.3 Attention mechanism in transformer architecture

```
class TransformerStyleGraphAttention(nn.Module):
   def __init__(self, feature_dim):
       super().__init__()
       self.query_transform = nn.Linear(feature_dim, feature_dim)
       self.key_transform = nn.Linear(feature_dim, feature_dim)
       self.value_transform = nn.Linear(feature_dim, feature_dim)
```

Initializes linear transformations for query, key, and value projections

```
    def forward(self, node_features, neighbor_features):
        Q = self.query_transform(node_features)
        K = self.key_transform(neighbor_features)
        V = self.value_transform(neighbor_features)

        attention_scores = torch.matmul(Q, K.transpose(-2, -1))
        attention_scores = attention_scores / math.sqrt(K.size(-1))

        attention_weights = F.softmax(attention_scores, dim=-1)
        attended_values = torch.matmul(attention_weights, V)

        return attended_values
```

Transforms input features into query, key, and value representations

Computes scaled dot-product attention_scores between query and keys

Applies softmax to get attention_weights and compute the weighted sum of values

Returns the attention-weighted node representations

This transformer-style attention in GNNs offers several advantages:

- *Scale and efficiency*—Like transformer models, this attention mechanism can efficiently process large neighborhoods by parallelizing computations.
- *Flexible feature learning*—Each attention head can specialize in capturing different types of relationships or patterns in the graph structure.
- *Interpretability*—The attention weights can provide insights into which neighbor relationships are most important for different tasks.

The connection to transformers also extends to how these models handle different scales of information. Just as transformers use positional encodings to capture sequence position information, some GNN architectures incorporate structural or positional encodings to capture graph topology. The following listing shows an example.

Listing 11.4 Including structural encoding to capture graph topology

```
class StructuralGraphTransformer(nn.Module):
    def __init__(self, feature_dim, num_heads):
        super().__init__()
        self.structural_encoding = nn.Embedding(max_degree, feature_dim)
        self.attention = MultiHeadGraphAttention(feature_dim, num_heads)

    def forward(self, node_features, neighbor_features, degrees):
        structural_features = self.structural_encoding(degrees)
        enhanced_features = node_features + structural_features

        return self.attention(enhanced_features, neighbor_features)
```

Initializes an embedding layer for structural encodings and multihead attention

Enhances node features with learnable structural information based on node degrees

Applies multihead attention using the structurally enhanced features

This convergence between GNNs and transformer architectures has important implications for the future of graph learning. As LLMs continue to advance, we can expect more cross-pollination of ideas between these domains. For instance, recent work has explored using LLMs to generate natural language descriptions of graph structures and using graph structures to enhance language understanding in LLMs.

11.6.4 Generalized update methods

The update phase of message passing is as important as aggregation but often receives less attention. The way we update node representations using the aggregated information can significantly impact a GNN's performance. Let's explore several powerful approaches to the update step.

Skip connections in GNNs

One of GraphSAGE's key innovations was the introduction of skip connections in the update phase. Similar to residual connections in traditional DL, skip connections in GNNs help preserve information across message-passing layers. Here's how GraphSAGE implements this concept.

Listing 11.5 `update` function in the GraphSAGE algorithm

```
class GraphSAGEUpdate(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.update_nn = nn.Linear(input_dim * 2, hidden_dim)

    def forward(self, node_features, aggregated_neighbor_features):
        combined = torch.cat([node_features, aggregated_neighbor_features],
        dim=1)

        updated = self.update_nn(combined)
        return F.relu(updated)
```

Initializes a neural network for transforming concatenated features

Concatenates node and aggregated neighbor features

Applies the nonlinear transformation and activation function

This concatenation-based update serves several purposes:

- *Information preservation*—By concatenating the node's previous state with aggregated neighborhood information, the network maintains access to the node's original features.
- *Feature separation*—The model can learn which aspects of the original node features and neighborhood information are most relevant for the task.
- *Gradient flow*—Skip connections provide additional paths for gradient flow during backpropagation, helping to train deeper GNN architectures.

Gated updates

Inspired by recurrent neural networks, *gated update* mechanisms provide finer control over how node representations are updated (see the next listing). These mechanisms are particularly valuable when nodes need to selectively incorporate neighborhood information.

Listing 11.6 Implementing gated updates

```
class GatedGraphUpdate(nn.Module):
    def __init__(self, feature_dim):
        super().__init__()
        self.update_gate = nn.Linear(feature_dim * 2, feature_dim)
        self.transform = nn.Linear(feature_dim * 2, feature_dim)
```

Initialize gate and transform networks for feature updating.

```
    def forward(self, node_features, aggregated_features):
        gate_input = torch.cat([node_features, aggregated_features], dim=1)
        update_gate = torch.sigmoid(self.update_gate(gate_input))

        combined = torch.cat([node_features, aggregated_features], dim=1)
        candidate = torch.tanh(self.transform(combined))

        return (1 - update_gate) * node_features + update_gate * candidate
```

Computes update gate values to control information flow

Generates candidate features through nonlinear transformation

Applies a gated update mechanism to combine old and new features

The gating mechanism allows the model to dynamically decide how much of the original node representation to preserve and how much new neighborhood information to incorporate. This is particularly useful in scenarios where some nodes should maintain more stable representations across layers, the relevance of neighborhood information varies across nodes, or the graph contains noisy or irrelevant connections.

JUMPING KNOWLEDGE NETWORKS

Another powerful update strategy involves maintaining representations from multiple layers and combining them adaptively. This approach, known as *jumping knowledge networks*, allows the model to capture multiscale structural information (listing 11.7).

Listing 11.7 Implementing jumping knowledge

```
class JumpingKnowledge(nn.Module):
    def __init__(self, feature_dim, num_layers):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=feature_dim,
            hidden_size=feature_dim,
            batch_first=True
        )

    def forward(self, layer_representations):
        stacked = torch.stack(layer_representations, dim=2)
        batch_size, num_nodes, num_layers, feature_size = stacked.shape
        reshaped = stacked.reshape(batch_size * num_nodes,
                                   num_layers, feature_size)
        output, _ = self.lstm(reshaped)
        last_output = output[:, -1, :]
        final_output = last_output.reshape(batch_size, num_nodes, -1)
        return final_output
```

Initializes long short-term memory (LSTM) for combining multilayer representations

Stacks representations from different layers, with layers as the third dimension

Reshapes to combine batch and nodes dimensions for LSTM processing

Processes node representations through LSTM, treating layers as time steps

Extracts features from the final layer

Reshapes back to separate batch and nodes dimensions in the final output

This approach offers several advantages. Different nodes can effectively use information from different numbers of hops away, and the model can combine local and global structural information. And by maintaining access to earlier layer representations, the model can avoid the loss of distinctive node features.

Practical considerations

When implementing these update mechanisms, we need to consider several factors:

- *Computational efficiency*—More complex update mechanisms generally require more computation time and memory.
- *Task requirements*—Different tasks may benefit from different update strategies.
- *Graph properties*—The structure and characteristics of the input graph can influence which update mechanism is most appropriate.

For example, in a citation network where papers need to balance their content with influences from cited works, a gated update mechanism may be most appropriate. In contrast, for a molecular graph where atomic properties are influenced by multiscale structural patterns, a jumping knowledge approach could be more suitable.

These update mechanisms represent different approaches to balancing the preservation of node-specific information with the incorporation of neighborhood context. In practice, the choice of update mechanism can be as important as the choice of aggregation strategy, and the two components should be designed to complement each other.

In chapter 12, we'll see how these update mechanisms can be combined with the aggregation strategies we discussed earlier to create powerful GNN architectures for specific applications. We'll explore how different combinations of these components perform on real-world tasks and provide practical guidance for choosing the right update mechanism for your specific use case.

11.7 The synergy of GNNs and LLMs

GNNs and LLMs are powerful paradigms in modern AI, each with strengths that become more potent when they are combined thoughtfully. As we have seen in this chapter, GNNs excel at processing structured graph data through message passing between nodes, capturing local neighborhood information, and global graph topology. They are particularly effective at tasks like node classification and link prediction because they can aggregate features from neighboring nodes and encode graph structures. However, GNNs typically struggle with processing rich textual information associated with nodes and edges.

LLMs, on the other hand, demonstrate remarkable capabilities in understanding and generating natural language text. They can process long sequences of text, capture semantic relationships, and even exhibit reasoning abilities. However, they are fundamentally designed to work with sequential data and don't naturally handle the complex structural relationships present in graphs.

LLMs can be combined with GNNs in three primary ways [13]:

- *LLMs as predictors*—In this approach, LLMs serve as the final component for making predictions or generating outputs. The graph structure can be encoded into a sequence format that LLMs can process, or the LLM architecture can be modified to handle graph structures directly. This approach is particularly powerful when the task requires combining structural understanding with complex reasoning or natural language generation. For example, in a KG-based question-answering system, a GNN might first process the graph structure, and then an LLM could generate natural language answers based on both the graph embeddings and the question context.

- *LLMs as encoders*—Here, LLMs are used to process textual information associated with nodes or edges, creating rich feature representations that are then passed to GNNs for structural processing. This approach uses LLMs' text understanding capabilities while maintaining GNNs' ability to process graph structures. For instance, in a scientific citation network, an LLM could encode paper abstracts into meaningful vectors, which a GNN could then use to model citation relationships and predict future citations.
- *LLMs as aligners*—This approach uses LLMs alongside GNNs, aligning their outputs through techniques like contrastive learning and mutual training. The two models work in parallel, with each handling its specialized domain (text or structure) and their outputs being combined or aligned for the final task. This is particularly useful in scenarios where maintaining separate but aligned representations of both textual and structural information is beneficial, such as in multimodal KGs where both the connection patterns and the textual content carry important information.

The key to successful integration lies in understanding the complementary strengths of each technology and designing architectures that use these strengths appropriately for the task at hand. As both GNNs and LLMs continue to evolve, we expect to see even more sophisticated ways of combining their capabilities to build more powerful and intelligent systems for KG applications.

Summary

- Graph representation learning automates feature engineering by transforming nodes and edges into dense vector representations. Its evolution spans three generations: traditional dimensionality reduction, word2vec-inspired approaches, and modern GNNs.
- Graph embeddings can be either positional (preserving global structure) or structural (capturing local patterns), each suited for different tasks.
- The selection of an embedding method should consider data characteristics, computational resources, and specific application requirements.
- The choice between transductive and inductive approaches depends on whether new nodes are expected to join the graph.
- Geometric spaces matter: some graph structures are better represented in non-Euclidean spaces, particularly for hierarchical data.
- The encoding–decoding process works like a translation system, converting graph data into vectors and then back to meaningful patterns.

- The encoder–decoder framework unifies different embedding approaches by separating graph encoding from property reconstruction.
- Shallow embeddings provide a simple but important baseline through their lookup-table approach to graph representation.
- Message passing in GNNs enables automatic feature discovery through iterative information aggregation from neighboring nodes.
- Multihead attention enables GNNs to process different types of node relationships in parallel, similar to transformers.
- Neighborhood normalization and attention mechanisms help handle varying neighborhood sizes and relationship importance.
- Skip connections and gated updates improve information flow across multiple GNN layers, preventing feature loss.
- LLMs complement GNNs by providing three integration paths: as predictors, encoders, or aligners for graph tasks.